WEEK 2 · LESSON 7 OF 24 · HANDOUT

NumPy: Fast Maths on Arrays

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=GB9ByFAIAH4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** freeCodeCamp: Data Analysis with Python (NumPy, Pandas, Matplotlib) — full course — https://www.youtube.com/watch?v=r-uOLxNrNk8

Your goal in this lesson: Create NumPy arrays, do maths on whole arrays at once, filter with conditions, and work with 2-D arrays — the format of almost all AI data.

Why NumPy?

A Python list is flexible but slow for maths. **NumPy** stores numbers in an **array**: one block of memory, all the same type. Maths on arrays runs in fast compiled code — often 50–100 times faster than a loop. Every AI library (Pandas, scikit-learn, PyTorch) is built on this idea.

```
import numpy as np

prices = np.array([7500, 3200, 1600, 800, 1000])
print(prices * 1.18)      # VAT on every price at once
print(prices.sum(), prices.mean(), prices.max())
```

No loop needed. Doing the same operation on every element at once is called **vectorisation**.

Creating arrays

```
a = np.array([2, 4, 6])
zeros = np.zeros(5)                 # [0. 0. 0. 0. 0.]
ones = np.ones(3)
steps = np.arange(0, 10, 2)          # [0 2 4 6 8]
grid = np.linspace(0, 1, 5)          # 5 evenly spaced values from 0 to 1
print(a.dtype, a.shape)              # int64 (3,)
```

Maths between arrays

Arrays of the same shape combine **element by element**:

```
quantity = np.array([4, 2, 6, 10, 5])
unit_price = np.array([7500, 3200, 1600, 800, 1000])
totals = quantity * unit_price
print(totals)                         # [30000  6400  9600  8000  5000]
print(totals.sum())                   # 59000
print(np.round(totals / totals.sum() * 100, 1))   # share of income in %
```

Indexing, slicing and filtering

```
print(totals[0], totals[-1])     # first and last
print(totals[1:3])               # items 1 and 2

big = totals > 8000              # a True/False array
print(big)                        # [ True False  True False False]
print(totals[big])               # only the big sales
print((totals > 8000).sum())     # how many are big -> 2
```

Filtering with a True/False array is called **boolean indexing**. It is the NumPy way to say *"give me the rows where…"*.

2-D arrays: rows and columns

AI data is usually a **table**: one row per example, one column per feature.

```
# columns: size_m2, bedrooms, age_years  (three houses)
houses = np.array([
    [120, 3, 10],
    [80, 2, 25],
    [200, 4, 3],
])
print(houses.shape)          # (3, 3) -> 3 rows, 3 columns
print(houses[0])             # first house
print(houses[:, 0])          # the size column
print(houses.mean(axis=0))   # average of each column
```

`axis=0` means *"go down the rows"* (one answer per column); `axis=1` means *"go across the columns"* (one answer per row).

Random numbers for simulations

```
rng = np.random.default_rng(42)          # a seeded random generator
dice = rng.integers(1, 7, size=10000)     # roll a die 10,000 times
print(dice.mean())                         # close to 3.5
heights = rng.normal(165, 8, size=1000)   # 1,000 simulated heights in cm
print(heights.mean().round(1), heights.std().round(1))
```

Simulation with random numbers is a powerful way to understand probability (Week 3).

Key terms

- **Array** — a NumPy grid of numbers of one type.
- **Vectorisation** — doing an operation on a whole array at once, without a loop.
- **Shape** — the size of each dimension, e.g. `(3, 3)`.
- **Boolean indexing** — selecting elements with a True/False array.
- **axis** — the direction of a calculation: `axis=0` down columns, `axis=1` across rows.

Remember this: NumPy arrays let you do maths on thousands of numbers in one line. Learn shape, vectorised maths, boolean filtering and `axis` — they appear in every AI library.

Name: _____ Date: _____

Activity

Task 1

Create an array of 7 daily temperatures in Kigali (invent them). Print the mean, the highest and how many days were above 26 °C.

Task 2

Convert the temperatures to Fahrenheit in one line (`F = C × 9/5 + 32`).

Task 3

Make a 2-D array with the marks of 4 students in 3 subjects. Print each subject's average (`axis=0`) and each student's average (`axis=1`).

Task 4

Simulate 10,000 coin flips with `rng.integers(0, 2, 10000)` and print the share of heads.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is vectorisation?
   - A. Doing an operation on a whole array at once, without a Python loop
   - B. Drawing arrows
   - C. Saving a file
   - D. Converting text to numbers
2. What does `np.array([1, 2, 3]) * 2` give?
   - A. An error
   - B. 12
   - C. [1, 2, 3, 1, 2, 3]
   - D. [2, 4, 6]
3. An array has shape (3, 4). What does that mean?
   - A. 3 rows and 4 columns
   - B. 4 rows and 3 columns
   - C. 3 numbers
   - D. 12 rows
4. What does `totals[totals > 8000]` return?
   - A. True/False values
   - B. The number 8000
   - C. The positions of big values
   - D. Only the elements greater than 8000
5. In a 2-D table of features, what does `.mean(axis=0)` give?
   - A. One overall number
   - B. The shape
   - C. The mean of each row
   - D. The mean of each column
6. Why use `np.random.default_rng(42)`?
   - A. To round numbers
   - B. To sort arrays
   - C. To create a seeded generator so results can be repeated
   - D. To get truly unpredictable numbers every time

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.